PixMath: A-to-Z Master Architecture, Specialized UI/UX & Implementation Blueprint

1. Executive Summary & Project Vision

PixMath is an industrial-grade, bidirectional software platform, Command-Line Interface (CLI), and interactive Web Studio that bridges pure algebraic mathematics and digital computer graphics. Inspired by mathematical art such as Hamid Naderi Yeganeh's "Solar Eruption" (where a 2000 x 1200 image is generated purely from nested trigonometric, exponential, and product series equations without a single brushstroke or 3D mesh), PixMath operates in two complementary directions:

- **The Forward Engine (Math to Image):** Accepts closed-form mathematical equations mapping continuous 2D coordinates (x, y) to (R, G, B) color channels, validates them through a compiler-level Abstract Syntax Tree (AST) security sandbox, transpiles them into OpenGL Shading Language (GLSL), and renders multi-million-pixel images in sub-milliseconds via GPU shaders.
- **The Reverse Engine (Image to Math):** Solves the inverse problem by taking an arbitrary digital image and extracting a compact, human-readable mathematical formula that reconstructs the image. It combines Implicit Neural Representations (Sinusoidal Representation Networks / SIREN and Kolmogorov-Arnold Networks / KANs) with evolutionary Symbolic Regression (PySR) and Pareto Frontier optimization.

2. Mathematical & AI Foundations

2.1 Forward Rendering Engine: Continuous Math to Discrete Pixel Matrices

Instead of rasterizing polygons, the Forward Engine treats an image as a continuous 2D domain where each pixel at column m and row n is normalized into Cartesian coordinates (x, y) in the range [-1, 1] and passed into a composite function f(x, y) → (R, G, B).

- **Parametric Trigonometric Compositions:** Complex organic textures (such as solar plasma, feathers, or waves) are synthesized by layering high-order sine and cosine polynomials, phase shifts, and frequency modulations. For example, structural boundaries can follow (x - A(k))^2 + (y - B(k))^2 = R(k)^2, where center shifts A(k) and B(k) are modulated by sin((pi / 2) * (k / c1)^7) and cos^16(c2 * k).
- **Signed Distance Functions (SDFs):** Sharp, anti-aliased geometric boundaries are defined using SDFs, which return the orthogonal distance from (x, y) to a surface (negative inside,

positive outside, zero on the boundary). For a circle of radius r, the SDF is f(x, y) = sqrt(x^2 + y^2) - r. Complex shapes are built using Boolean algebra:
    - **Union:** min(SDF_A, SDF_B)
    - **Intersection:** max(SDF_A, SDF_B)
    - **Subtraction:** max(SDF_A, -SDF_B)
- **Procedural Noise & Domain Warping:** To introduce organic irregularity into rigid geometric polynomials, continuous Perlin or Simplex noise functions warp the coordinate space prior to evaluation: f(x + noise(x, y), y + noise(x, y)).
- **8-Bit RGB Clamping:** Continuous outputs are mapped into integer RGB values (0 to 255) using smooth analytical clamping functions or hardware shader saturation.

2.2 Reverse Extraction Engine: Image to Closed-Form Equations

Converting a discrete pixel matrix I (Height x Width x 3) back into a concise mathematical formula requires balancing spatial fidelity against formula length:

- **Approach A — 2D Fast Fourier Transform (FFT) & Fourier Epicycles:** Decomposes an image into overlapping 2D sine and cosine frequencies, or traces Canny edge contours using complex epicycles Z(t) = Sum(c_k * exp(i * k * t)). While fast and parallelizable, pure FFT methods produce massive, dense polynomials and suffer from Gibbs ringing artifacts around sharp edges.
- **Approach B — 2D Gaussian Splatting (2DGS) & Quadratic Gaussian Splatting (QGS):** Represents an image as localized 2D Gaussian primitives defined by position mu, covariance matrix Sigma = R * S * S^T * R^T, RGB color, and opacity, or deformable quadric surfaces v^T * Q * v = 0. While achieving ultra-fast spatial compression, it outputs an unstructured parameter matrix rather than a single symbolic formula.
- **Approach C — The PixMath Core Pipeline (SIREN / KAN + PySR Symbolic Regression):**
    - **Sinusoidal Representation Network (SIREN):** Standard ReLU Multi-Layer Perceptrons suffer from spectral bias and blur high-frequency image details. PixMath trains a coordinate MLP using periodic sine activations: Phi_i(x) = sin(W_i * Phi_{i-1}(x) + b_i). Because derivatives of sine waves are phase-shifted sine waves, SIREN preserves crisp edges and fine textures.
    - **Kolmogorov-Arnold Networks (KANs):** As an alternative or complement to SIREN, KANs place learnable 1D spline activation functions on network edges rather than fixed activations on nodes, naturally decomposing multi-variable mappings into sums of univariate functions that convert directly into symbolic trees.
    - **Evolutionary Symbolic Regression (PySR):** Once the neural representation maps (x, y) → (R, G, B), PySR runs evolutionary genetic programming (tournament selection, subtree crossover, operator mutation) over populations of ASTs to distill the network into a closed-form algebraic expression.

2.3 Quality Metrics & Pareto Frontier Selection

A 10,000-page equation that achieves perfect accuracy is useless to humans. PixMath optimizes a dual objective:

- **Visual Fidelity Loss:** Combines pixel-wise L2 loss (measured via Peak Signal-to-Noise Ratio, PSNR) with structural perception loss (Dissimilarity Structural Similarity Index, D-SSIM):
  L_rec = lambda * L_2 + (1 - lambda) * L_D-SSIM
- **AST Complexity Score:** Calculated by traversing the equation's AST and summing all operator, variable, and constant nodes.
- **Automated Pareto Elbow Selector:** As PySR evaluates generations of formulas, PixMath maps Accuracy (PSNR/SSIM) vs. Complexity (AST Node Count) along a Pareto frontier curve and computes the curvature derivative to automatically select the equation at the "Pareto elbow"—delivering maximum visual quality with the shortest, most readable formula.

3. Industrial Microservices & Hybrid BFF Architecture

PixMath decouples lightweight synchronous user interactions from long-running asynchronous mathematical workloads using an Event-Driven Microservices topology fronted by a Hybrid Backend-for-Frontend (BFF) API Gateway.

3.1 Strategic Protocol Division (REST + GraphQL + WebSockets)

- **REST API Layer (Binary Ingestion & CDN Delivery):** Reserved exclusively for multipart-form binary image uploads (`POST /api/v1/jobs/upload`), chunked streaming, presigned S3/Cloudflare R2 URL generation, and HTTP caching (`ETag`, `Cache-Control`) for rendered high-resolution PNG exports.
- **GraphQL API Layer (Structured Metadata & AST Slicing):** Powered by Strawberry/Apollo, GraphQL handles relational queries for user history, render configurations, and selective equation inspection. Clients can request specific slices of an equation (for example, `equation(maxTerms: 50)`) and utilize `@defer` and `@stream` directives so lightweight metadata arrives immediately while heavy AST nodes stream progressively.
- **WebSockets & GraphQL Subscriptions (Real-Time Evolution Stream):** Because SIREN training and PySR genetic search can run from seconds to minutes, workers publish live telemetry to a Redis PubSub bus. GraphQL Subscriptions push real-time generation updates, live PSNR/SSIM curves, and intermediate LaTeX formulas directly to the UI without client polling.

3.2 Event-Driven RabbitMQ Compute Pipeline

- **Message Contract:** Contains `job_id`, presigned R2 source image URL, target resolution, AST complexity budget, and evolutionary hyperparameters.
- **Backpressure Control (`prefetch_count = 1`):** Because symbolic regression jobs vary dramatically in duration, setting RabbitMQ's consumer `prefetch_count` strictly to `1` prevents the broker from flooding a single worker with queued jobs while other nodes sit idle.
- **Publisher Confirms & Dead-Letter Exchange (DLX):** Asynchronous Publisher Confirms guarantee messages are persisted to disk before notifying the gateway. If a worker crashes with an Out-Of-Memory (OOM) error on a pathological equation, the unacknowledged message routes automatically to a Dead-Letter Exchange (DLX), where the `dlx-monitor` service retries the job with reduced AST depth limits or alerts the client via WebSocket.

4. Compiler-Level Security & Defense-in-Depth AST Sandboxing

Executing user-submitted math strings in Python via `eval()` or naive `SymPy.parse_expr()` exposes servers to Remote Code Execution (RCE) and Denial-of-Service (DoS) attacks. PixMath enforces a 3-layer security firewall:

1. **Layer 1 — Strict `ast.NodeVisitor` Whitelisting:** User strings are parsed into Python Abstract Syntax Trees and inspected node-by-node. Only explicitly whitelisted mathematical constructs (`ast.Expression`, `ast.BinOp`, `ast.UnaryOp`, `ast.Constant`, `ast.Name` restricted to `x`, `y`, `t`, `pi`, `e`, and `ast.Call` restricted to safe trigonometric/SDF functions like `sin`, `cos`, `tan`, `asin`, `acos`, `exp`, `sqrt`, `abs`, `min`, `max`, `noise`) are permitted. Any attribute access (`__getattr__`), imports, or lambdas immediately trigger a security rejection.
2. **Layer 2 — Structural Depth & Node Volume Caps:** To block exponential CPU/stack bombs such as `2**(2**(2**(2**32)))`, the validator traverses the tree prior to evaluation and enforces hard caps on maximum vertical tree depth (e.g., max depth = 32) and total AST node count (e.g., max nodes = 2,000).
3. **Layer 3 — Ephemeral 512 MB Linux cgroup Sandboxes:** Execution runs inside network-isolated Docker containers or microVMs (such as E2B/Modal) locked down with Linux `cgroups` (512 MB RAM max, 1 vCPU, zero outbound internet access, and a strict 15-second execution timeout for forward rendering). For ultra-fast CPU fallback, expressions are evaluated via C++ `ExprTk`.

services/forward-worker/app/sandbox/ast_validator.py

import ast

ALLOWED_NODES = {

```
    ast.Expression, ast.BinOp, ast.UnaryOp, ast.Constant, ast.Name, ast.Call,
    ast.Add, ast.Sub, ast.Mult, ast.Div, ast.Pow, ast.Mod, ast.USub, ast.UAdd
}
ALLOWED_NAMES = {'x', 'y', 't', 'pi', 'e'}
ALLOWED_FUNCTIONS = {
    'sin', 'cos', 'tan', 'asin', 'acos', 'atan', 'atan2',
    'exp', 'log', 'sqrt', 'abs', 'min', 'max', 'clamp', 'noise'
}
class SecurityASTValidator(ast.NodeVisitor):
    def __init__(self, max_depth=32, max_nodes=2000):
        self.max_depth = max_depth
        self.max_nodes = max_nodes
        self.current_depth = 0
        self.node_count = 0
    def generic_visit(self, node):
        self.node_count += 1
        if self.node_count > self.max_nodes:
            raise ValueError(f"Security Alert: Node limit exceeded ({self.max_nodes})")


        if type(node) not in ALLOWED_NODES:
            raise SecurityError(f"Forbidden AST Node: {type(node).__name__}")
```

```
        self.current_depth += 1
        if self.current_depth > self.max_depth:
            raise ValueError(f"Security Alert: AST Depth limit exceeded ({self.max_depth})")

        super().generic_visit(node)
        self.current_depth -= 1
    def visit_Name(self, node):
        if node.id not in ALLOWED_NAMES:
            raise SecurityError(f"Forbidden Variable Name: {node.id}")
        self.generic_visit(node)
    def visit_Call(self, node):
        if not isinstance(node.func, ast.Name) or node.func.id not in ALLOWED_FUNCTIONS:
            raise SecurityError("Forbidden Function Call")
        self.generic_visit(node)
```

5. Hardware Acceleration & High-Speed Serialization

5.1 GPU Shader Transpilation (AST to GLSL)

Evaluating a 4096 x 4096 image requires 16.77 million coordinate evaluations per channel. Instead of CPU-bound Python loops constrained by the Global Interpreter Lock (GIL), PixMath transpiles the validated AST directly into an OpenGL Shading Language (GLSL) fragment shader:

- **Safe GLSL Mathematical Wrappers:** The transpiler wraps division, square roots, and logarithms in safe guard functions (e.g., `safe_sqrt(v) = sqrt(max(v, 0.0))` and `safe_div(a, b) = a / (abs(b) < 1e-7 ? 1e-7 : b)`) so domain singularities never crash the GPU pipeline.

- **Dual GPU Execution Strategy:**
    - **Client-Side WebGL2 / WebGPU:** The API Gateway sends the transpiled GLSL shader string to the React Web Studio so the user's local GPU renders interactive 60 FPS previews at zero cloud compute cost.
    - **Server-Side Headless EGL (`ModernGL`):** For CLI batch jobs and high-resolution downloadable PNG exports, headless worker containers initialize display-free EGL contexts (`backend="egl"`) to render directly in framebuffers without an X11 or Wayland server.

```
// Generated GLSL Fragment Shader Output

#version 330 core

out vec4 FragColor;

in vec2 uv; // Mapped x, y in [-1.0, 1.0]

uniform float t;

float safe_div(float a, float b) {
    return a / (abs(b) < 1e-7 ? 1e-7 : b);
}

float safe_sqrt(float v) {
    return sqrt(max(v, 0.0));
}

void main() {
    float x = uv.x;
    float y = uv.y;

    // Transpiled AST Expression
    float r = sin(10.0 * safe_sqrt(x*x + y*y) - t);
```

```
    float g = cos(8.0 * atan(y, x) + t);

    float b = safe_div(x, y + 1.2);


    FragColor = vec4(clamp(r, 0.0, 1.0), clamp(g, 0.0, 1.0), clamp(b, 0.0, 1.0), 1.0);
}
```

5.2 Serialization Architecture Matrix

During PySR genetic evolution, workers mutate millions of AST structures. PixMath selects serialization formats based on memory and latency demands:

Format	Processing Mechanism	Memory & Speed Profile	PixMath Architectural Role
FlatBuffers	Zero-copy deserialization via binary vtables and memory offsets.	Exceptional; zero heap allocation or garbage collection spikes.	Caching and streaming millions of intermediate PySR AST mutation trees across 512 MB workers.
Protocol Buffers	Binary schema IDL unpacked into strongly typed memory objects.	High network speed, moderate decoding allocation.	Internal microservice contracts between API Gateway, RabbitMQ, and Worker nodes.
JSON	Full text parsing into dynamic object graphs.	High memory overhead and slower parsing.	External GraphQL API payloads and browser client state inspection.
LaTeX Strings	Formatted mathematical typesetting syntax.	Presentation-only; expensive to re-parse.	Final human-readable display in the Web Studio UI and exported PDF/Markdown reports.

6. Zero-Cost Cloud Stitching & 512 MB RAM Survival Guide

To host PixMath for $0/month without hitting memory walls or bandwidth fees, the platform stitches together specialized permanent free tiers:

6.1 Free-Tier Cloud Deployment Map

- **Frontend Web Studio:** Hosted on **Cloudflare Pages** (unlimited static bandwidth, 500 builds/month).
- **API Gateway (Hybrid REST + GraphQL):** Deployed on **Koyeb** Free Tier (512 MB RAM, 0.1 vCPU, always-on Docker container with zero cold-start sleep).
- **Forward & Reverse Math Workers:** Deployed on **Render** Free Docker Services (750 free hours/month) coupled with local CLI worker execution support.
- **Message Broker: CloudAMQP** (*Little Lemur* plan: 1M messages/month, 20 concurrent connections).
- **Database, Auth & Realtime PubSub: Supabase** (500 MB PostgreSQL with `pgvector` for AST similarity search, 50k MAU Auth, and Realtime WebSockets).
- **Object Storage (Images & Exports): Cloudflare R2** (10 GB free storage, 1M writes, 10M reads/month, and zero egress bandwidth fees).
- **Edge Equation Cache: Cloudflare Workers + D1** (caches popular pre-computed equations at the network edge).

6.2 Surviving the 512 MB RAM Limit on Reverse Workers

Importing `torch`, `sympy`, and `pysr` simultaneously consumes over 250 MB of baseline RAM before a single image is processed. PixMath prevents Out-Of-Memory (OOM) crashes using three engineering techniques:

1. **Lazy Module Loading (`importlib`):** Workers start as lightweight consumers (~30 MB RAM). Heavy libraries (`torch`, `pysr`) are dynamically imported inside isolated function scopes only at the exact second a phase begins.
2. **Explicit Tensor Deallocation & Forced GC:** As soon as the SIREN/KAN model finishes training and outputs its coordinate grid predictions, the worker deletes the model and optimizer objects (`del model, optimizer`), clears caches, and calls `gc.collect()` to reclaim memory before starting PySR.
3. **Distributed MapReduce Population Sharding:** Instead of evolving 20 large populations on one machine, the Reverse Engine shards small AST populations across stateless worker batches, stores surviving FlatBuffers ASTs in Supabase/Redis, and merges the Pareto frontier asynchronously.

7. Specialized UI/UX Experience ("PixMath Studio" & CLI)

Because PixMath is built for software engineers, mathematical artists, and AI researchers, its UI/UX combines the precision of a code IDE (like VS Code / Shadertoy) with the visual fluidity of a creative design tool (like Figma / Desmos).

7.1 Visual Identity & Aesthetic Direction

- **Theme:** Deep Obsidian Math-Lab Dark Mode (`#0B0F19` background, `#111827` elevated glass panels) with neon mathematical syntax accents—Cyan (`#06B6D4`) for spatial coordinates, Amber/Gold (`#F59E0B`) for trigonometric operators, and Emerald (`#10B981`) for Pareto-optimal nodes.
- **Typography:** `JetBrains Mono` or `Fira Code` with programming ligatures for equation editing, paired with `KaTeX` for instantaneous 60 FPS LaTeX math typesetting and `Inter` for clean UI controls.

7.2 Core Interactive Studio Workspaces

1. **Split-Screen Bi-Directional Canvas & Formula IDE:**
    - **Left Pane (The Symbolic Editor):** Features live AST syntax highlighting, bracket matching, and inline error squiggles if a user types an un-whitelisted function or exceeds tree depth.
    - **Hover-to-Highlight X-Ray Inspector:** When a user hovers their cursor over a sub-expression in the equation (for example, the solar flare arch product term or an SDF circle primitive), the WebGL2 canvas on the right dynamically isolates and highlights the exact pixels governed by that sub-tree!
    - **Interactive Constant Scrubbers:** Holding `Alt` and dragging over any number in the equation turns that constant into a live slider, morphing the WebGL2 shader in real time at 60 FPS.
2. **The Interactive Pareto Frontier Explorer (Reverse Mode):**
    - When extracting an equation from an uploaded image, a live scatter/line chart plots **Equation Complexity (AST Nodes)** on the X-axis against **Visual Accuracy (PSNR / SSIM)** on the Y-axis as WebSockets stream new generations from PySR.
    - Users can click any dot along the Pareto curve—from a 12-node minimalist sketch equation up to a 500-node photorealistic equation—and watch the reconstructed image and LaTeX formula morph instantaneously.
3. **Region-of-Interest (ROI) Brush & Layer Decomposition:**
    - Users can paint a mask over a specific part of an uploaded photo (like the fiery loop of a solar flare) to allocate higher AST complexity weight to that region while keeping the background equation simple.
4. **First-Class Terminal UX (`pixmath` CLI):**
    - Built with Python `Typer` and `Rich`, featuring ASCII/Unicode Braille live terminal previews of rendered equations, animated progress bars for SIREN/PySR convergence, and one-command local studio launch (`pixmath studio`).

8. Future Features & Modern Trend Roadmap

To future-proof PixMath as a flagship open-source and commercial platform, the architecture supports six cutting-edge extensions:

1. **4D Spatio-Temporal Video Equations (`f(x, y, t) -> RGB`):** Introduce time variable `t` into the AST and SIREN/KAN pipeline so users can render infinite, resolution-independent mathematical animations or compress short video loops into a single time-varying algebraic formula.
2. **Client-Side WebGPU Compute Shaders & WASM Pyodide:** Offload not just forward rendering, but also lightweight SIREN training directly onto the user's browser GPU via **WebGPU** compute shaders, enabling zero-server offline extraction.
3. **LLM-Guided Symbolic Priors (Neuro-Symbolic Seeding):** Before starting PySR's genetic search, pass the target image through a Vision-Language Model (VLM) to predict structural mathematical priors (e.g., detecting radial symmetry, spiral geometry, or periodic grids) and seed the initial PySR population with matching SDF and polar coordinate primitives.
4. **Audio-Reactive Mathematical Shaders (`f(x, y, fft_audio)`):** Bind live microphone or audio frequency bins into equation variables so mathematical art pulses and deforms in sync with music for VJing and creative coding.
5. **CAD & Differentiable SVG/STEP Export:** Convert extracted level-set SDF equations (`f(x, y) = 0`) directly into infinite-resolution parametric CAD splines or SVG paths for laser cutting, 3D printing, and vector design.
6. **"Git for Math" Collaborative Formula Registry:** Allow users to fork, diff, and merge AST mathematical equations visually—comparing two equations by overlaying their AST diffs alongside an A/B curtain slider of their rendered outputs.

9. Complete Monorepo Layout & 18-Week Implementation Guide

9.1 Production Monorepo Directory Structure

pixmath/

├── apps/

│ ├── cli/ # Typer + Rich terminal client (render, extract, studio)

│ ├── web-studio/ # React + Tailwind + WebGL2/WebGPU live shader & Pareto UI

│ └── api-gateway/ # Hybrid BFF: FastAPI (REST upload) + Strawberry (GraphQL/WS)

├── services/

```
│   ├── forward-worker/       # ast.NodeVisitor sandbox + AST-to-GLSL transpiler + ModernGL/EGL
│   ├── reverse-worker/       # PyTorch SIREN/KAN trainer + PySR genetic engine + FlatBuffers
│   └── dlx-monitor/          # Telemetry service watching RabbitMQ Dead-Letter Exchanges
├── libs/
│   ├── schemas/              # FlatBuffers (ast_tree.fbs) & Protobuf (job_contract.proto)
│   └── math-core/            # Shared SDF primitives, Simplex noise, PSNR & D-SSIM metrics
├── infra/
│   ├── docker/               # 512MB cgroup-constrained Dockerfiles & headless EGL configs
│   └── compose.yaml          # Local dev stack (RabbitMQ, Redis PubSub, Postgres, MinIO)
└── tests/
    ├── unit/                 # Math accuracy & GLSL transpiler unit tests
    ├── integration/          # End-to-end REST → RabbitMQ → Worker → WebSocket tests
    └── redteam/              # AST security fuzzing & exponential bomb penetration tests
```

9.2 Step-by-Step 4-Phase Engineering Roadmap

Phase 1: Mathematical Foundations & Forward MVP (Weeks 1–4)

- **Week 1 (AST Firewall):** Implement `services/forward-worker/app/sandbox/ast_validator.py` using Python's `ast.NodeVisitor`. Enforce strict node whitelisting, variable name checks (`x`, `y`, `t`), max tree depth (`32`), and max node volume (`2000`). Build the Red-Team test suite in `tests/redteam/` to verify immunity against `__import__`, `__subclasses__`, and power bombs.
- **Week 2 (CPU Baseline & Math Core):** Implement `libs/math-core/` with vectorized NumPy evaluators, 2D SDF primitives (circle, box, torus, smooth-min), and Simplex noise domain warping.
- **Week 3 (AST-to-GLSL Compiler):** Build `services/forward-worker/app/transpiler/ast_to_glsl.py` to recursively compile

validated Python AST nodes into GLSL `#version 330 core` fragment shaders with safe division and square-root guards.

- **Week 4 (Headless GPU & CLI v0.1):** Configure `headless_gl.py` using `ModernGL` with `backend="egl"` inside Docker (`libegl1-mesa-dev`), and wire up `pixmath render --eq "..." --out art.png` in the Typer CLI.

Phase 2: The Neural-Symbolic Reverse Engine (Weeks 5–9)

- **Week 5 (SIREN & KAN Models):** Implement `services/reverse-worker/app/siren/network.py` (periodic sine MLP with principled weight initialization `w0 = 30.0`) and `kan/spline_net.py` using PyTorch.
- **Week 6 (Perceptual Optimization Loop):** Build the coordinate training pipeline in `libs/math-core/pixmath_core/metrics/psnr_ssim.py` optimizing the composite loss `L_rec = lambda * L_2 + (1 - lambda) * L_D-SSIM`.
- **Weeks 7–8 (PySR Symbolic Distillation):** Connect `pysr_engine/symbolic.py` to sample coordinate-color mappings from the trained SIREN/KAN network and evolve closed-form algebraic ASTs using custom trigonometric, exponential, and SDF operators.
- **Week 9 (Pareto Elbow Selector):** Implement `pareto/selector.py` to compute the Pareto frontier curve, identify the optimal accuracy-to-complexity elbow, and export clean SymPy/LaTeX strings.

Phase 3: Event-Driven Infrastructure & Hybrid BFF (Weeks 10–14)

- **Week 10 (RabbitMQ Broker):** Configure RabbitMQ in `infra/compose.yaml` with `fast-render-queue` and `deep-symbolic-queue`, enforcing `prefetch_count = 1`, Publisher Confirms, and Dead-Letter Exchanges (DLX).
- **Week 11 (Hybrid REST + GraphQL Gateway):** Build `apps/api-gateway/app/rest/upload.py` for multipart image streaming to Cloudflare R2 and `app/graphql/schema.py` for querying jobs, equations, and AST slices.
- **Week 12 (WebSockets & Live Pareto Streaming):** Integrate Redis PubSub with GraphQL Subscriptions (`app/ws/pubsub.py`) so every new PySR generation streams live PSNR, AST node count, and formula previews to connected clients.
- **Weeks 13–14 (FlatBuffers & Protobuf Serialization):** Compile `libs/schemas/flatbuffers/ast_tree.fbs` and `protobuf/job_contract.proto` to eliminate JSON memory spikes during worker AST caching.

Phase 4: 512 MB Hardening, Web Studio & Cloud Launch (Weeks 15–18)

- **Week 15 (PixMath Web Studio UI):** Build the React + Tailwind + WebGL2 Studio (`apps/web-studio/`) featuring the Split-Screen Formula IDE, Hover-to-Highlight X-Ray Inspector, Constant Scrubbers, and Interactive Pareto Chart.

- **Week 16 (512 MB Memory Engineering):** Add `importlib` lazy loading, explicit PyTorch tensor deallocation (`del`, `gc.collect()`), and Distributed MapReduce population sharding across stateless workers.
- **Week 17 (Container Hardening & Penetration Testing):** Lock down `infra/docker/` containers with Linux `cgroups` (`--memory=512m --cpus=1 --network=none` for sandboxed evaluators) and run full Red-Team fuzzing.
- **Week 18 (Zero-Cost Cloud Stitching Deployment):** Deploy the React UI to Cloudflare Pages, image storage to Cloudflare R2, database/realtime to Supabase, broker to CloudAMQP, API Gateway to Koyeb, and math workers to Render.